kprime Storefront — Build Plan

Medusa v2 backend (existing) + fresh Next.js App Router storefront.
Pakistan only · PKR · COD only · English only · Guest checkout only.

At a glance

§	Section	What it settles
1	Scope	16 routes, guest-only COD checkout, reviews in. Accounts, OTP, messaging, Urdu, free delivery → v2
2.1	Filtering	Native option filtering on v2.19; price + facet counts in the Next.js server layer. Specs are **options, not metadata**
2.1.1	Category tree	Self-referencing table, leaf-only assignment, flat single-segment URLs
2.1.2	Per-category filters	Derived from data, ordered by config. Renders only at ≥25% coverage
2.2	Checkout	Guest only. Name, phone, address required. Synthetic email = identity key. Verification is a phone call
2.3	Design tokens	Navy text, amber action, red sale, green success, cream page
2.4	Reviews	Flat + one merchant reply, moderated, verified buyer via phone
3	Pages	16 routes and how each renders
4	Components	63 components across `ui/`, `layout/`, `shared/`, `page/`

5	Backend config	8 admin tasks. Category tree + option spelling **before importing**
6	Phases	10 phases, 6–9 weeks solo
6.1	Build sequence	Numbered 1–36 session order to a live home page
7	Working method	One component per session, read every diff
8	Open questions	What's left before Phase 2

Five things most expensive to get wrong

1. **Option spelling** (§2.1) — a typo silently splits a filter or creates a phantom one
2. **Category handles** (§2.1.1) — globally unique, locked before import; renaming breaks shared links
3. **`normalizePhone()`** (§2.2) — the identity key; unnormalised forks a customer's history in two
4. **The synthetic email format** (§2.2) — freeze it; it's how v2 accounts claim guest history
5. **`ProductCard`** (§6.1 step 16) — five pages inherit it, and it's most of what mobile visitors see

---

1. Scope

In v1: multi-level categories · faceted filtering · search · product detail with variants · guest cart · guest COD checkout (name, phone, address) · order confirmation · `/track` · product reviews · static policy pages.

Not in v1 — each easier to add later than to carry unfinished:

Deferred	Why it's safe to defer
Online payment	COD only for now

Multi-region / multi-currency	Single region
Customer accounts	Nothing in v1 depends on them; they layer on top (§2.2)
Order history by phone	Without OTP nothing proves ownership. `/track` covers the real need
OTP / **SMS** / **WhatsApp**	Verification is a call, not a feature (§2.2)
Free delivery threshold	A shipping option rule in admin, not architecture (§5.1)
Urdu	English only
Wishlists, coupons, loyalty	Additive
Returns portal	Handle over WhatsApp initially

---

2. Four decisions before writing components

2.1 How filtering works

Backend confirmed on **Medusa v2.19.0**. Option-value filtering landed in v2.16, so most of what you need is native. No search engine for v1.

Native on `/store/products`: `category_id` · `collection_id` · `tag_id` · `type_id` · `option_value_id` · `option_id` · `q` · `order` · `limit`/`offset`. Build the sidebar from `/store/product-options`.

Not native: price range (prices are calculated at query time) · metadata filtering (JSONB, not queried by the store API) · facet counts.

→ Model filterable specs as product options, not metadata.
Metadata is a dead end for filtering. An option with a **single value** adds no variants — a shirt that's always cotton gets `Fabric: Cotton`,

variant count unchanged, and it's filterable for free.

⚠ **Naming convention — decide before importing.** Options belong to individual products, so `Cotton` on product A and product B are separate records. The sidebar groups by string and passes every matching ID. Spelling must be identical catalogue-wide:

- `Color`, never `Colour`/`color`/`COLOR`
- `Red`, never `red`/`Bright Red`/`Red.`
- Title Case throughout, no trailing spaces
- Units in the value, one form: `128GB`, not `128 GB` or `128gb`

Verify grouping against real data in Phase 2 before building the whole sidebar.

Where each filter runs

Medusa	Next.js server layer
Category, subcategory, colour, size, shade, storage, fabric, material (`option_value_id`), brand (tag), keyword (`q`), sort (`order`)	**Price range**, **facet counts**

You can't let Medusa paginate *and* post-filter, or counts go wrong. For a category, fetch the full result set with `fields` trimmed to what `ProductCard` needs, cache with `unstable_cache`, then filter and paginate in server memory. ~500 bytes per trimmed product → 500 products is ~250KB held server-side, never shipped to the browser. Comfortable to 500–1000 per category.

This is **server-layer** filtering inside an RSC, not client-side filtering. Never ship the catalogue to the browser — it kills mobile performance, SEO, and shareable filter URLs.

Put it all behind one function: `lib/data/products.ts → searchProducts(params)`. Adding MeiliSearch later rewrites that one file, not the listing page, sidebar and URL handling.

Attribute mapping

Category	Variant options	Single-value specs	Tags
Electronics	Colour, Storage	RAM, Screen Size, Warranty	brand
Cosmetics	Shade, Size/ Volume	Skin Type, Formulation	brand
Dresses	Size, Colour	Fabric, Sleeve, Fit, Occasion	brand
Kitchen	Colour, Capacity	Material, Pieces in Set	brand
Bed sheets	Bed Size, Colour	Material, Thread Count, Pieces	brand

2.1.1 Category tree

One table. `product_category` self-references via `parent_category_id`, no depth limit. A subcategory is a row with a parent. Products link many-to-many. ~45 rows (5 top-level × ~8).

⚠ **Parent categories don't inherit their children's products.** `category_id = Electronics` returns only products explicitly assigned to Electronics, not ones in Mobiles.

Assignment rule: leaf only. For a parent page, fetch with `include_descendants_tree=true`, collect descendant IDs, pass the array. The descendants call runs an extra query per record — fetch the tree once, cache with `unstable_cache`, resolve from memory.

Handles are globally unique. No `accessories` under both Electronics and Cosmetics — use `mobile-accessories` and `makeup-accessories`. Lock the full list before importing.

Set `rank` on every category, or nav order follows creation date.

Flat URLs — `/categories/[handle]`, single segment. This is what makes the tree restructurable: inserting `Electronics > Computers > Laptops` where it was `Electronics > Laptops` changes no URL and needs no redirects. Breadcrumbs still show the full hierarchy, they

just link to short URLs.

Build the nav recursively — render whatever depth the tree returns. Costs nothing now; means adding a level post-launch is an admin task, not a header rewrite.

Depth: 3 levels max for v1, varying per branch. A *content* ceiling, not a code one. At 3 levels the desktop mega menu uses level 2 as column headings with level 3 beneath; mobile uses nested accordions.

2.1.2 Per-category filters

Medusa has no concept of category attributes — categories are a naming tree and own nothing. Every product carries its own options, so subcategories can have completely different attributes with no schema work. **`config/filters.ts` is effectively your category schema.**

Derive filters from the data; use config only for order and exclusion. The page already fetches the full result set, so it can see which option titles are present. Render those, ordered by config.

```
export const filterOrder = [
  "Price", "Brand", "Color", "Size", "Material",
  "Storage", "RAM", "Shade", "Bed Size", "Thread Count",
]
export const filterHidden = ["Warranty", "Country of Orig
```

New subcategories need no config change, filters never render with zero values, and new options added in admin appear automatically. Trade-off: a typo no longer just splits a filter — it creates an entirely new one. Another reason to lock the option sheet.

Coverage threshold: render a filter group only if it covers **≥25%** of the current result set. RAM covers ~15% of Electronics and hides; 100% of Laptops and shows. Price and Brand always clear the bar. Counts are free — the result set is already in server memory.

Parent pages naturally end up with only cross-cutting filters (price, brand, colour); leaf pages get their specific specs. That falls out of the

approach without extra work.

Semantics: OR within a group, AND across groups. When a group is active, products lacking that option are excluded — filtering RAM on Electronics correctly drops every blender.

Taxonomy rule: if two products in a leaf can't sensibly be compared on the same filters, they belong in different leaves. Laptop vs blender fails. This usually means Electronics needs a middle level:

```
Electronics
├── Laptops           → RAM, Storage, Processor, Screen Si
├── Mobiles           → Storage, RAM, Screen Size, Battery
├── Home appliances   → Wattage, Capacity, Warranty
└── Accessories       → Type, Compatibility
```

2.2 Phone-first guest checkout

Guest only in v1. No accounts, no login, no OTP, no messaging.
Required: name, phone, province, city, street address. Email optional.
No postal codes — Pakistan's aren't reliable; zones run off city (§5.1).

Medusa requires an email on every cart. Many COD customers don't use one, so it's synthesised rather than requested.

The identity rule — three pieces, all cheap now and expensive to retrofit:

1. **`normalizePhone()` runs at the API boundary**, not in the component. Accepts `03xxxxxxxxx`, `+923xxxxxxxxx`, `00923…`, spaces, dashes → returns `923001234567`. Nothing else touches a raw phone string.
2. **Guest cart email is always the phone-derived synthetic address** — `{normalised}@nomail.kprime.pk`, even when a real email is supplied. The real one goes to `cart.metadata.contact_email`. Deterministic mapping means one phone = one customer record. ⚠ **Freeze this format** — it's the key that lets v2 accounts claim guest history.
3. **The normalised phone is written onto the Customer record**

during checkout. `phone` is a native field, so it becomes a real admin column.

What this gets you free. Medusa creates a Customer with `has_account: false` on guest checkout. Since the email is phone-derived, **that record is your per-phone ledger** — every order, address, delivery outcome and lifetime value. Search the phone in Admin → Customers. No custom module, works from your first order. It's also what the verified-buyer check on reviews queries (§2.4).

This is why the phone lives on the Customer record and not `order.metadata` — metadata is JSONB and unqueryable, same constraint as §2.1.

Contact phone and `shipping_address.phone` may legitimately differ (gifts). Let them, and use the **contact** phone as identity.

COD verification — manual. All COD orders accepted, no cap, no advance payment. The number is verified by a **phone call before dispatch**, not an OTP. At launch volumes a 30-second call confirms intent to pay, which is what predicts RTO; an OTP only confirms the phone exists.

Set `metadata.phone_verified = false` at placement, flip on the call. Gives you an admin filter and, after a few hundred orders, a called-vs-uncalled RTO comparison.

Reversal cost if RTO climbs: short-code SMS OTP ~Rs 4.70–4.80/ message, Rs 10,000 minimum package (1-year validity), needs NTN + CNIC + app-specific PTA authorization taking weeks. WhatsApp is *not* cheaper here — Meta's authentication rate in Pakistan is ~Rs 21.50, ~4.5× the SMS route. Build `src/modules/otp/` behind a two-provider interface (real + console stub) if and when needed.

Order lookup: `/track` takes order number **and** phone, rate-limited. The order number is the shared secret.

⚠ **Never build phone-only lookup on the storefront.** Phone numbers aren't secret and `03XX` is enumerable. An open endpoint hands a stranger's address, purchase history and prices to anyone with their number. Rich per-phone lookup stays admin-side.

The confirmation page is the only receipt. No email, no SMS — a customer who closes the tab has nothing. It must carry the order number prominently, items, total, delivery estimate, your WhatsApp number, and a "screenshot this" prompt. Keep copy soft ("we've received your order, we'll be in touch if we need anything") so the confirmation call doesn't contradict it.

Accounts in v2. Nothing above changes. Registration uses stock `emailpass`; the synthetic email is the join key. ⚠ Medusa does **not** merge guest orders into a new account — it creates a second Customer record. Fix with a subscriber on `order.placed` that reassigns orders to a registered customer with a matching normalised phone, rather than a one-time merge (the same person can check out as a guest again next month).

In v1, no "Login" or "Create account" anywhere. A dead link promising a feature is worse than its absence.

2.3 Design tokens

Navy brand, amber action, red sale, green success, on a light page. Every colour has exactly one job — that discipline is what makes a small palette read as designed rather than sparse.

Role	Hex	Used for
Page	`#F6F4EF`	Page background (cream, not white)
Card	`#FFFFFF`	Cards, panels, modals
Brand	`#0F1E3D`	All text, headings, prices, logo, header/ footer, icons, active nav
Brand light	`#1E3A6B`	Hover on navy surfaces
Action	`#F2A007`	Add to cart, primary CTA — **always with `#1A1408` text**
Action hover	`#D98906`	
Sale	`#C2410C`	Discount badges, savings

Success	`#15803D`	In stock, order confirmed — **never a CTA**
Muted	`#6B7280`	Breadcrumbs, labels, strikethrough prices, metadata
Line	`#E6E2DA`	Borders, dividers

```
colors: {
  cream:  "#F6F4EF",
  paper:  "#FFFFFF",
  brand:  { DEFAULT: "#0F1E3D", light: "#1E3A6B" },
  action: { DEFAULT: "#F2A007", hover: "#D98906", ink: "#
  sale:    "#C2410C",
  success: "#15803D",
  muted:   "#6B7280",
  line:    "#E6E2DA",
}
```

Non-negotiable

1. **Navy replaces black for text.** Never `#000`, `#333`, `text-gray-900`.
2. **Amber only ever means "act on this."** Buttons and nothing else, ~2% of the page.
3. **Dark text on amber, never white.** White on `#F2A007` is 2.1:1 and fails WCAG; `#1A1408` is 8.1:1. The most common contrast mistake in ecommerce.
4. **Green is success only.** A green button beside a green "in stock" label makes both meaningless.
5. **Sale red is distinct from brand.** Discount visibility drives COD conversion.
6. **Light page, always.** Supplier images of mixed quality look far worse on dark backgrounds.

Logo. `KARKHANO` small above `PRIME` large, ~1:3 cap height. KARKHANO all caps with ~0.35em letterspacing so its width matches PRIME's. PRIME tight and heavy. One geometric sans (Poppins, Outfit, or Manrope). Three files before Phase 1: full-colour on light, reversed on navy, and a square `P` mark in cream on navy for favicon and WhatsApp.

Still to decide in Phase 0: font pairing, type scale, spacing base (4px or 8px), radius, density. Collect 2–3 reference screenshots.

2.4 Reviews

Medusa v2 has no reviews module. Custom code: model, service, link to product, store routes, admin moderation routes. 4–6 days, built **after** checkout works.

- **Flat, not nested.** One review + optional merchant reply. Keep a `parent_id` column but don't render deeper. Threading breaks moderation, breaks pagination, and is unreadable at 360px.
- **Reviews, not Q&A.** Rating + text, one per phone per product. Discussion threads are a different feature.
- **Verified buyer via the phone's Customer record** — "does a delivered order exist under this phone containing this product". Restricting to verified buyers solves most spam structurally.
- **Moderation mandatory.** Default `pending`; nothing renders until approved.
- **Denormalise the aggregate.** Store `average_rating` and `review_count` on the product, recompute on approve/delete. Otherwise a 24-product grid fires 24 aggregate queries.
- **Rating filter and sort run in the server layer** — same as price, same reason. `RatingFilter` is a real component; sort gains "Highest Rated".
- **Caching:** first page of reviews in the static payload, `revalidateTag` on approval, further pages client-side.
- **SEO payoff:** `AggregateRating` in JSON-LD puts stars in Google results.
- **Defer photo reviews.** Prompt for reviews on the order detail page — there's no confirmation message to put a link in.

---

3. Pages

16 routes. Mobile-first — assume 80%+ mobile traffic.

#	Route	Purpose	Rendering
1	`/`	Home	Static + revalidate
2	`/categories/[handle]`	Listing + filters	Dynamic (searchParams)
3	`/products/[handle]`	Product detail + reviews	Static + revalidate
4	`/search`	Search, same filter UI	Dynamic
5	`/collections/[handle]`	Merchandised sets (Sale, New In)	Static + revalidate
6	`/cart`	Cart	Dynamic
7	`/checkout`	Contact → address → shipping → COD → review	Dynamic
8	`/order/confirmed/[id]`	The only receipt	Dynamic
9	`/track`	Lookup by order number **and** phone	Dynamic
10–15	`/about` `/contact` / `faq` `/shipping-and-delivery` `/returns-and-refunds` / `privacy` `/terms`	Policy	Static
16	`not-found.tsx` `error.tsx` `loading.tsx`	System	—

Routing

- **Three dynamic segments:** `[handle]` for categories, products,

collections; `[id]` for the confirmation. Handles are globally unique, so one path segment is enough — that flatness is what lets the category tree be restructured after launch.

- **Static + revalidate** — home, product, collections. `generateStaticParams` over handles, `revalidate` on a timer, `revalidateTag` on review approval.
- **Dynamic** — category, search, cart, checkout, confirmation, track. They depend on `searchParams` or the cart cookie. Category is dynamic because filter state lives in the URL, which is what makes filtered views shareable and the back button correct.
- **Two route groups:** `(shop)` for the full shell, `(checkout)` for a stripped layout — logo, step indicator, trust strip. Removing the exits is the point. Parentheses keep the group name out of the URL.
- **One data layer.** Everything fetches through `lib/data/*`; no component touches the SDK directly. That's what makes the MeiliSearch swap a one-file change.

---

4. Components

63 components. `ui/` = primitives, `layout/` = shell, `shared/` = 3+ pages, `page/` = one page.

4.1 Structure

```
components/
├── ui/                          # 11 primitives — Phase 0
│   Button, Input, Select, Checkbox, RadioGroup, Badge,
│   Skeleton, Drawer, Modal, Toast, Accordion
│
├── layout/                      # 11 — the shell, Phase 1
│   Header, CategoryMegaMenu, MobileNav, SearchBar, CartB
│   AnnouncementBar, Footer, Breadcrumbs, Container, Trus
│   WhatsAppFloatButton
│
```

```
├── shared/                    # 9 — used on 3+ pages
│   ProductCard, ProductGrid, ProductRail, PriceDisplay,
│   QuantityStepper, CartSummary, OrderItemsList, EmptySt
│
└── page/                      # 32 — one page only
    ├── home/                  # 6  → page 1
    │   HeroCarousel, CategoryGrid, PromoBannerPair, Bran
    │   NewsletterSignup, HomeSkeleton
    │
    ├── catalog/               # 11 → pages 2, 4, 5
    │   CategoryHeader, SearchResultHeader, FilterSidebar
    │   PriceRangeFilter, CheckboxFilterGroup, ColorSwatc
    │   RatingFilter, ActiveFilterChips, SortDropdown, Pa
    │
    ├── product/               # 10 → page 3
    │   ProductGallery, GalleryThumbnails, MobileGalleryS
    │   ProductTitleBlock, VariantOptionSelector, AddToCa
    │   StockIndicator, DeliveryEstimateBox, ProductTabs,
    │
    ├── review/                # 5  → page 3
    │   ReviewSummary, ReviewList, ReviewCard, ReviewForm
    │
    ├── cart/                  # 4  → page 6
    │   CartLineItem, CartLineItemMobile, CartDrawer, Emp
    │
    ├── checkout/              # 9  → page 7
    │   CheckoutStepper, ContactStep, ShippingAddressStep
    │   ShippingMethodStep, PaymentStep, OrderReviewStep,
    │   OrderSummaryPanel
    │
    ├── order-confirmed/       # 1  → page 8
    │   OrderConfirmationHero
    │
    └── track/                 # 2  → page 9
        TrackOrderForm, OrderStatusTimeline
```

Rules

- **Imports run one way.** `page/` → `shared/` → `layout/` → `ui/`.

Never import across two `page/` folders — if `checkout/` needs something from `cart/`, that thing belongs in `shared/`.

- **Promote, don't duplicate.** A component starts in its page folder; the moment a second page needs it, move it to `shared/`.
- **`catalog/` covers three pages** — category, search, collections are the same page with different headers, which is why the folder is named for the function.
- **Colours and spacing come from `tailwind.config`, never inline.** A hex code inside a component means a token is missing.

4.2 Page → component map

Two layouts. Build the shell once; only the middle changes.

Shop shell — `app/(shop)/layout.tsx`, pages 1–6 and 8–15: `AnnouncementBar` → `Header` (contains `CategoryMegaMenu`, `SearchBar`, `CartButton`) → `MobileNav` → `Container` → *page* → `Footer` → `WhatsAppFloatButton`

Checkout shell — `app/(checkout)/layout.tsx`, page 7 only: Logo → `CheckoutStepper` → *page* → `TrustStrip`. No nav, no search, no footer links.

Page	Components
1 `/`	`HeroCarousel` · `CategoryGrid` · `ProductRail` ×3 · `PromoBannerPair` · `BrandStrip` · `TrustStrip` · `NewsletterSignup`. Loading: `HomeSkeleton`
2 `/categories/[handle]`	`Breadcrumbs` · `CategoryHeader` · `ActiveFilterChips` · `SortDropdown` · `ProductGrid` · `PaginationControls` · `EmptyState`. Filters render twice from the same children: `FilterSidebar` (desktop) and `FilterDrawer` (mobile), both containing `PriceRangeFilter`, `CheckboxFilterGroup`, `ColorSwatchFilter`, `RatingFilter`

3 `/products/[handle]`	`Breadcrumbs` · `ProductGallery` + `GalleryThumbnails` / `MobileGallerySwiper` · `ProductTitleBlock` · `StarRating` · `PriceDisplay` · `VariantOptionSelector` · `StockIndicator` · `QuantityStepper` · `AddToCartButton` · `DeliveryEstimateBox` · `ProductTabs` · `StickyMobileBuyBar` · reviews block (`ReviewSummary` → `ReviewList` → `ReviewCard` → `MerchantReply`, plus `ReviewForm`) · `ProductRail`. Adding to cart opens `CartDrawer` + `Toast`
4 `/search`	Same as page 2 with `SearchResultHeader` instead of `CategoryHeader`, no breadcrumbs
5 `/collections/[handle]`	`Breadcrumbs` · `CategoryHeader` · `SortDropdown` · `ProductGrid` · `PaginationControls`. No facets — merchandised sets don't need them
6 `/cart`	`CartLineItem` / `CartLineItemMobile` (each with `QuantityStepper`, `PriceDisplay`) · `CartSummary` · `TrustStrip` · `Button` · `EmptyCart`
7 `/checkout`	`ContactStep` · `ShippingAddressStep` (contains `ProvinceCitySelect`) · `ShippingMethodStep` · `PaymentStep` · `OrderReviewStep` · `PlaceOrderButton` · `OrderSummaryPanel` (reuses `OrderItemsList`, `CartSummary`). Accordion steps driven by `CheckoutStepper`
8 `/order/confirmed/[id]`	`OrderConfirmationHero` · `OrderItemsList` · `CartSummary` · `DeliveryEstimateBox` · WhatsApp block · "screenshot this" · `Button`
9 `/track`	`TrackOrderForm` · `OrderStatusTimeline` · `OrderItemsList` · carrier + tracking number + URL · `EmptyState`

10–15	`Container` + prose. `/contact` adds a form and WhatsApp block; `/faq` uses `Accordion`
16	`not-found.tsx` (fired by `notFound()`) · `error.tsx` at root and around checkout · `loading.tsx` per dynamic route

---

5. Backend configuration

All in the Medusa admin. Do it **before** building checkout or nothing works.

1. **Region** — Pakistan, PKR, confirm tax settings
2. **Sales channel + publishable key** — the storefront key must be linked to the channel
3. **Stock location + fulfillment set** — Peshawar, one set ("Delivery")
4. **Service zones** — see §5.1. Zones are *price tiers* holding many cities, not one per city
5. **Shipping options** — Standard and Express per zone, `manual` provider, flat rate
6. **Payment provider** — enable `manual` (this is COD)
7. **Category tree** — build the real taxonomy ⚠ before importing products
8. **Option titles and values** — lock the spelling per §2.1, then create a reference sheet of every allowed title and value per category ⚠ before importing products

Reviews are the one piece that isn't admin config — a custom backend module (§2.4), Phase 7.

5.1 Shipping zones

Model: `Fulfillment Set → Service Zone → Geo Zones (cities)`, options attached to the zone. The service zone is a **price tier**, not a city. Four zones and ~7 options rather than 150 zones and

300 options — one price change updates 40 cities, and the customer experience is identical.

Couriers: TCS and Leopards, booked manually. Options stay **courier-agnostic** — the customer picks Standard or Express, admin picks the carrier at dispatch. One zone map covers both: take the *worse* case per city and price at the *higher* of the two rates.

Zone	Cities	Standard	Express
Local	Peshawar	TBD	TBD
Metro	Islamabad, Rawalpindi, Lahore, Karachi, Faisalabad, Multan, Gujranwala, Sialkot, Hyderabad, Quetta	TBD	TBD
Other cities	remaining courier-served	TBD	TBD
Remote	AJK, Gilgit-Baltistan, interior Balochistan	TBD	not offered

No free delivery threshold in v1. Every order pays the zone rate. Adding one later is an admin rule, not a storefront change — decide before you start marketing.

Delivery estimates live in the shipping option name — `Standard Delivery (3–5 days)`, `Express Delivery (1–2 days)`. Admin-editable, renders straight through, no code.

⚠ **City must be a dropdown, never free text.** Geo zone matching is on the city string. A customer typing `pindi` matches nothing and sees *zero* shipping options with no obvious error — a silent checkout dead-end.

To keep the admin as single source of truth, add `src/api/store/shipping-cities/route.ts`: reads geo zones from the fulfillment module, returns cities grouped by province. `ProvinceCitySelect` populates from it. ~40 lines, and new cities added in the dashboard appear in checkout automatically.

Checkout ordering constraint: shipping options resolve against the address, so the address step must complete *before* the shipping method step renders.

Manual fulfillment: tracking numbers are entered by hand and the carrier differs per order, so store carrier name alongside `tracking_number` and `tracking_url`, and surface both on `/track` and in the Brevo "order shipped" email.

---

6. Build phases

Each phase ends with something you can open in a browser and check.

Phase	Scope	Time	Done when
0	SDK client, `lib/data/*`, design tokens, primitives	2–3d	`/dev/styleguide` renders every primitive in every state
1	Header, mega menu, mobile nav, footer, breadcrumbs, container	3–4d	Nav reflects your real category tree
2	Category page, grid, card, full filter system, sort, pagination, search	1–1.5w	Filters work, survive refresh, and are shareable as URLs
3	Gallery, variants, specs, related, sticky mobile bar	3–4d	Variant switching updates price, image and stock
4	Cart — add, update, remove, persist, drawer	2–3d	Cart survives a browser restart; quantity edge cases behave

5	Checkout through order placement (guest only)	1–1.5w	A real order in admin with correct address, shipping and total — **and** a second order from the same phone lands under the same customer record
6	Confirmation page, `/track`, order status	1–2d	An order is retrievable by number + phone; the confirmation page works as a standalone receipt
7	Reviews — module, routes, moderation, components	4–6d	A verified-phone review appears only after approval, and the average updates on card and detail
8	Static pages, SEO — metadata, OG, sitemap, robots, JSON-LD incl. `AggregateRating`	3–4d	
9	Hardening — mid-range Android on 3G/4G, Lighthouse, bad-input validation, out-of-stock races, Brevo emails	3–5d	

Realistic total for a careful solo build: 6–9 weeks. Checkout and filtering are over half.

6.1 Sequence to a working home page

One component per session, in this order. Each is buildable because everything it imports exists.

Setup (not components)

1. `tailwind.config.ts` — palette from §2.3, type scale, spacing base, radius
2. `lib/sdk.ts` — Medusa JS SDK client with the publishable key
3. `lib/data/products.ts` — `searchProducts()`, `getProduct()`
4. `lib/data/categories.ts` — cached category tree
5. `app/(shop)/layout.tsx` — empty shell, filled at step 26

Nothing below works without step 1.

Primitives (6–13) — `Button`, `Input`, `Select`, `Checkbox`, `Badge`, `Skeleton`, `Drawer`, `Accordion`. `RadioGroup`, `Modal` and `Toast` aren't needed for home — defer to Phases 4–5. If using shadcn, these collapse to ~2 sessions: generate, then rewrite `buttonVariants` and the CSS variables to your tokens. The stock variants and neutral grays violate §2.3 as shipped.

Shared (14–18) — `PriceDisplay` → `StarRating` → `ProductCard` → `ProductGrid` → `ProductRail`. ⚠ **`ProductCard` is the highest-leverage component in the build.** Five pages inherit it and it's most of what a mobile visitor sees. Get image ratio, title truncation and the discount badge right here.

Layout (19–29) — `Container` → `Breadcrumbs` → `AnnouncementBar` → `SearchBar` → `CartButton` → `CategoryMegaMenu` → `MobileNav` → `Header` (composes 22–25) → `TrustStrip` → `Footer` → `WhatsAppFloatButton`. `CategoryMegaMenu` and `MobileNav` need the cached tree from step 4.

Home (30–36) — `HeroCarousel` → `CategoryGrid` → `PromoBannerPair` → `BrandStrip` → `NewsletterSignup` → `HomeSkeleton` → assemble `app/(shop)/page.tsx`.

36 sessions to a live home page, of which only 6 are home-specific. That's why the category page in Phase 2 is mostly assembly — its filters are the only genuinely new work.

Session prompt shape:

> Create `components/shared/ProductCard.tsx`. Props: `product`

> (Medusa `StoreProduct`). Shows image, title (2-line truncate), `PriceDisplay`, `StarRating`, and a `Badge` when `compare_at_price` exists. Use `Badge` from `components/ui`, colours and spacing from `tailwind.config` — no inline hex. Mobile-first, 2 columns at 360px.

File path, imports, constraint. Then read the diff before accepting.

---

7. Working method with Claude Code

The point of this rebuild is that you understand the result. That only happens if you enforce:

1. **One component per session.** Name the file path in the prompt.
2. **Reference the tokens.** "Use `Button` from `components/ui`, spacing from `tailwind.config`, match the layout in this screenshot."
3. **Read every diff before accepting.** If you can't explain it, it doesn't go in.
4. **Run it after each step.** Broken states are cheap now, expensive later.
5. **Commit per component**, with a message you'd understand in a month.
6. **Keep `CLAUDE.md` current** — stack, conventions, folder structure, the filtering approach, the phone-first rule. It's read at the start of every session and is the main lever on quality.
7. **Never say "make it better" or "clean this up."** Unbounded prompts produced the last codebase. Specific, bounded tasks with a visible result.

---

8. Open questions

Settled

- Filtering — native option filtering on v2.19, price + facet counts in the server layer
- SKU count — under 200 per category at launch; ~3–5× headroom, MeiliSearch not needed
- Category tree structure — self-referencing table, leaf-only assignment, cached tree
- Shipping zones — 4 tiers, courier-agnostic, express everywhere but Remote
- COD risk controls — manual confirmation call, logged to `metadata.phone_verified`
- Customer accounts — deferred to v2; preserve the synthetic email format and `normalizePhone()`
- Order history lookup — `/track` by order number + phone only
- Reviews — flat + one merchant reply, moderated, verified buyer via phone
- Language — English only
- Free delivery threshold — none in v1

Still open

- ☐ Option title/value list per category, and the locked spelling convention
- ☐ Final category tree contents and the globally-unique handle list — **lock before importing**
- ☐ Actual rates per zone from the TCS and Leopards sheets (take the higher of the two)
- ☐ Review moderation policy — who approves, how fast, what gets rejected
- ☐ Confirmation-call script and where it sits in dispatch
- ☐ Hosting (affects caching and image strategy)